

A Small-Scale Experimental Study of the Muon Optimizer and Newton-Schulz Iterations for Neural Network Training

Initial Proposal | COE202 Mathematical Foundations of AI

1. Motivation

Optimization is a central problem in modern deep learning. AdamW is widely used because it is stable, adaptive, and relatively easy to tune. Recently, the Muon optimizer has attracted attention as a promising alternative, especially for hidden layers of neural networks. Unlike AdamW, which applies coordinate-wise adaptive updates, Muon applies momentum followed by a matrix orthogonalization step. This step is approximated by Newton-Schulz iterations, linking the optimizer to polar decomposition, matrix norms, and numerical linear algebra.

This project asks: Can Muon achieve faster or more stable convergence than AdamW on small neural networks, and how sensitive is its performance to the Newton-Schulz iteration design? Existing discussions of Muon often focus on large-scale language models, while direct controlled comparisons between Muon and AdamW on small networks remain limited. Therefore, this project narrows the setting to MNIST, Fashion-MNIST, and a CIFAR-10 subset, using MLPs and small CNNs where repeated experiments are feasible.

2. Preliminary Work

I have reviewed the main ideas behind AdamW, Muon, and Newton-Schulz iterations. AdamW maintains first- and second-moment estimates of stochastic gradients and applies decoupled weight decay:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t g_t^T \\ \theta_t &= (1 - \alpha \lambda) \theta_{t-1} - \alpha * m_{\hat{t}} / (\sqrt{v_{\hat{t}}} + \epsilon). \end{aligned}$$

Muon uses a different matrix-geometric idea. For a matrix parameter W_t with gradient G_t , it first computes a momentum matrix $M_t = \beta M_{t-1} + (1 - \beta)G_t$. Ideally, Muon would update using the polar direction $\text{Polar}(M_t) = U_t V_t^T$, where $M_t = U_t \Sigma_t V_t^T$. In practice, it avoids exact SVD and instead approximates this direction through a small number of Newton-Schulz iterations:

$$X_{j+1} = a X_j + (b X_j X_j^T + c (X_j X_j^T)^2) X_j, j = 0, \dots, q - 1.$$

The iteration count q and coefficients (a, b, c) affect the trade-off between approximation quality, computational cost, and stability. In particular, practical Muon implementations use coefficients such as $(3.4445, -4.7750, 2.0315)$, while more standard polynomial coefficients such as $(15/8, -5/4, 3/8)$ can also be studied.

3. Proposed Work

The project will combine mathematical explanation with controlled experiments. First, I will present AdamW and Muon update rules and explain why Newton-Schulz iterations are useful: they replace expensive SVD-based orthogonalization with repeated matrix multiplications, which are efficient on modern hardware. Second, I will compare AdamW and Muon on small image-classification tasks using the same model architecture, data split, training budget, and random seeds.

Dataset	Model	Optimizers / Variants	Main Metrics
MNIST	MLP	AdamW vs Muon	loss, accuracy, time
Fashion-MNIST	MLP / small CNN	AdamW vs Muon	loss, accuracy, stability
CIFAR-10 subset	small CNN	AdamW vs Muon	loss, accuracy, time, memory
Selected task	MLP or CNN	Muon with q in $\{1,2,3,5\}$; coefficient ablation if time permits	speed, stability, cost

The main metrics will be training loss versus epoch, training loss versus wall-clock time, validation accuracy, time-to-threshold loss, stability across five random seeds, epoch time, and peak GPU memory. Hyperparameters such as learning rate and weight decay will be tuned using a fixed search budget. For Muon, I will additionally test the number of Newton-Schulz steps q and, if time permits, compare the practical coefficient choice $(3.4445, -4.7750, 2.0315)$ with $(15/8, -5/4, 3/8)$.

To reduce confounding factors, I will use two protocols. Protocol A removes BatchNorm where possible and uses bias-free matrix layers to isolate optimizer effects. Protocol B follows the common engineering use of Muon, applying Muon to hidden matrix parameters while using AdamW for biases, classifier heads, or parameters that are not suitable for Muon. This separation allows the final report to distinguish scientific comparison from practical engineering usage.

4. Expected Outcome

I expect Muon to reduce training loss faster than AdamW in settings where matrix-valued hidden layers dominate the model, especially during early training. However, I do not assume that faster loss reduction will always imply better final validation accuracy. AdamW may remain more robust or easier to tune on very small models or noisy settings.

I also expect the Newton-Schulz design to matter. A small number of iterations, such as $q = 2$ or $q = 3$, may provide a good trade-off between approximation quality and runtime. Too few iterations may produce unstable updates, while too many may increase computation without clear benefit. The coefficient choice may also influence stability and convergence speed.

The final result will be a concise empirical and mathematical comparison of AdamW and Muon. Even if Muon does not consistently outperform AdamW, mixed or negative results would still be meaningful because they would clarify when matrix-orthogonalized updates are useful, when AdamW remains preferable, and how optimizer design interacts with model size, data complexity, and computational cost.

References

- [1] Keller Jordan. Muon: An Optimizer for Hidden Layers in Neural Networks, 2024.
- [2] Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization, 2019.
- [3] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization, 2015.
- [4] G. Kim and M. Oh. Convergence of Muon with Newton-Schulz, ICLR 2026.
- [5] Moonshot AI. Muon / Moonlight Technical Report, 2025.
- [6] Nicholas J. Higham. Functions of Matrices: Theory and Computation, 2008.